Formalisme du modèle eGenBrain v4

Dossier d'implémentation et d'auditabilité

24 octobre 2026

Statut épistémique

Ce document définit le contrat cible de v4 et distingue quatre statuts : (i) hérité et exécuté par v3, (ii) implémenté dans la couche v4, (iii) testé seulement dans un micro-monde, (iv) encore ouvert. La présence d'une définition dans ce document ne sera pas prise comme preuve de sa réalisation : chaque composant doit être relié au code et à des tests exécutables dans `preuves_eGenBrain_v4.json`. La théorie `theorie_de_maillage_eGenBrain_0.tex` est le cadre de référence ; `eGenBrain_1.tex`, `eGenBrain_v2.tex`, le formalisme v3 et la théorie DDD sont conservés comme sources historiques.

Chapitre 1

Objectif et exigences

L'objectif de v4 est d'étendre v3 sans effacer sa provenance : (a) conserver l'ASCR, MG persistant, MGR/MGR2, l'arbitrage v3 et les traces ; (b) implémenter un maillage stratifié permettant de représenter faits typés, observations, règles multi-prémisses, opérateurs, provenance et certificats ; (c) faire un moteur général de composition de règles et d'opérateurs, sans coder des réponses de tâches individuelles ; (d) exposer une interface/CLI pour ajouter des primitives, inspecter les preuves, exécuter des requêtes et comparer le raisonnement maillé au moteur v3 seul ; (e) sauvegarder les sorties réelles.

Chapitre 2

Définition opérationnelle du maillage v4

Définition 2.1 (Maillage stratifié exécutable)**.** *Un état de maillage est*

$$\mathcal{M}_4 = (V, O, q, E, H, \tau, \pi, w, \kappa, \Delta, \Gamma, \Sigma, \mathcal{A}),$$

où V est l'ensemble d'identités canoniques, O l'ensemble d'occurrences/contextes, $q : O \to V$ l'identité canonique, E les relations binaires orientées typées, H les règles/hyperrelations multi-prémisses, τ les contraintes de types, π la provenance, w les supports/poids d'observation, κ les signatures observationnelles, Δ les transitions d'état autorisées, Γ les opérateurs typés, Σ les certificats, et $\mathcal{A}$ l'état des activations/requêtes. Chaque sous-ensemble absent est explicitement déclaré absent ; il n'est pas simulé par une arête binaire.

2.1 Faits, valeurs et types

Un fait est un tuple

$$f = (s, p, o, \tau_s, \tau_p, \tau_o, c, \pi, \rho),$$

où s, p, o sont sujet, prédicat, objet (l'objet peut être une valeur), τ_* désigne leurs types, c le contexte, π une chaîne de provenance et $\rho \in \{observed, derived, assumed\}$ le statut épistémique. Les faits observés et dérivés ne sont pas confondus. Une valeur numérique est représentée exactement (décimal/rationnel quand approprié) avec unité/dimension ; les nombres sérialisés en flottant binaire ne servent pas de preuve arithmétique.

Une occurrence conserve sa surface, sa position, son document source et son identifiant. Les empreintes sont indexées par un protocole d'observation et ne remplacent ni le type ni la provenance. Les signatures sont de l'identification/audit, pas des preuves sémantiques.

2.2 Règles et hyperrelations

Une règle est un tuple $r = (P_r, C_r, \chi_r, \pi_r)$ avec un ensemble fini de motifs de prémisses P_r, une ou plusieurs conclusions C_r, des variables partagées et contraintes de type χ_r, et une provenance. Une substitution θ rend r applicable seulement si toutes ses prémisses sont présentes et que toutes les contraintes sont satisfaites. La projection d'une règle conjonctive vers des arêtes indépendantes est interdite si elle change la sémantique.

2.3 Opérateurs et composition

Un opérateur $g \in \Gamma$ a une signature de type entrée/sortie, une fonction d'évaluation déterministe, des préconditions, une version et une provenance de déclaration. Les opérateurs arithmétiques standards peuvent être utilisés comme primitives générales lorsqu'ils sont explicitement disponibles ; les faits du domaine (par exemple facteur d'unité) doivent être appris/importés ou

observés et ne sont pas codés dans le moteur par tâche. Une composition $g_n \circ \cdots \circ g_1$ est admise si les types et dimensions de toutes les sorties/entrées s'accordent.

Définition 2.2 (Certificat de dérivation)**.** *Un certificat σ associe à chaque conclusion dérivée la liste des prémisses, identifiants des règles ou opérateurs, substitutions, signatures de types, valeurs intermédiaires, chaîne de provenance et empreinte du résultat. Une réponse de composition est publiée uniquement si le certificat est vérifiable par le validateur correspondant.*

Chapitre 3

Architecture et pipeline complet

1. Entrée : texte naturel ou requête structurée ; conserver l'entrée exacte et son empreinte.
2. Analyse : extraire des observations/faits candidats sans consulter la réponse attendue ; noter version et ambiguïtés du parseur.
3. Activation : déterminer les nœuds/faits pertinents à partir des requêtes et signatures ; tracer les éléments retenus/rejetés.
4. ASCR/v3 : exécuter le moteur de retrieval historique comme baseline ; son score de pertinence est un signal de recherche, pas une preuve de vérité.
5. MGR/MGR2 : reconstruire des maillages transitoires depuis un état neuf, avec provenance de chaque relation.
6. Inférence v4 : unifier les faits typés avec les règles multi-prémisses et les signatures d'opérateurs ; effectuer une clôture bornée et reproductible, avec détection des cycles et limites explicites.
7. Validation : contrôler les types, unités, cohérence des règles, provenance et certificat. Une arête observée ou un chemin lexical n'implique pas une causalité ou une conclusion.
8. Arbitrage : priorité à une dérivation certifiée conforme à la requête ; ensuite réponse reconstruite de v3 avec trace ; sinon abstention explicitement motivée. Les politiques et seuils doivent être affichés.
9. Persistance : enregistrer réponse brute, statut, trace, candidats, chemins, faits/règles utilisés, certificat, erreurs, timing et hash avant le scoring avec les références.
10. Évaluation : seulement après avoir persisté l'inférence, comparer aux réponses de référence ; garder séparés réussite d'exécution et exactitude.

Chapitre 4

Opérations sémantiques

4.1 Composition relationnelle

La transivité n'est pas implicite : elle doit être fournie par une règle de type $R(x, y) \wedge R(y, z) \Rightarrow R(x, z)$, limitée à un prédicat déclaré transitif. La symétrie n'est pas implicite non plus : $R(x, y)$ n'autorise pas $R(y, x)$ sans règle.

4.2 Calculs dimensionnels

Le total d'une quantité et d'un prix unitaire requiert : valeur de quantité, unité d'entrée, facteur de conversion/échelle fourni comme primitive, unité élémentaire, prix par unité, opérateur multiplication et compatibilité dimensionnelle. Le moteur ne contient aucune branche spéciale pour « œufs », « douzaine » ou une réponse attendue. Une chaîne illustrative est

$$3\,\text{dozen} \times (12\,\text{egg/dozen}) = 36\,\text{egg}, \qquad 36\,\text{egg} \times (0.50\,\text{USD/egg}) = 18\,\text{USD}.$$

Le facteur 12 doit être une primitive chargée dans le maillage, la quantité et le prix proviennent de l'observation d'entrée, et le 18 ne doit pas être enregistré comme cible ou réponse d'entraînement.

4.3 Généralisation

Les niveaux G0 (copie exacte), G1 (recombinaison de faits/règles vus), G2 (composition d'une combinaison non observée avec opérateurs connus), G3 (induction de règle) et G4 (transfert de domaine) sont rapportés séparément. Une exécution qui passe un test G2 n'établit pas G3/G4.

Chapitre 5

Correspondance héritée et extension réelle

Concept	**v2/v3 hérité**	**exigence v4**
MG / E	multigraphe de tokens et relations d'observation	faits typés, valeurs, provenance de relation conservée
ASCR	récupération/recherche d'expériences candidates	reste baseline ; ne certifie pas la vérité
MGR/MGR2	graphes transitoires reconstruits	restent isolés ; les dérivations v4 ont un espace distinct
H	pas de moteur général multi-prémisses démontré dans v3	moteur de règles déclaratives et unification
Γ	pas de composition dimensionnelle générale démontrée	opérateurs versionnés et typés
Σ	traces de chemins et provenance historiques	certificat de dérivation vérifiable, distinct d'une trace de retrieval
κ	empreintes d'observation DDD/canonique	protocole d'empreinte et contexte persistés
Interface	Studio v3	commandes et endpoints pour faits, règles, requêtes et inspection

Chapitre 6

Plan d'évaluation et garde-fous

Les jeux de test doivent être indépendants des réponses exactes et les réponses attendues rester cachées au moteur. Le split d'évaluation garde séparés les faits primitifs, règles, exemples de dérivation et combinaisons cibles. Tests ciblés : (A) composition A–B–C avec règle explicite ; (B) inversion non autorisée devant s'abstenir ; (C) transitivité avec prémisses A–B, B–C, C–D ; (D) addition 2+4 avec opérateur générique et sans exemple 2+4 ; (E) abstraction animale : abstention si aucune règle n'autorise Tiger ; (F) analogie : l'évaluation doit différencier une vraie règle structurale d'une simple similarité de noms ; (G) problème des œufs avec seule primitive d'échelle et prix par unité, réponse exacte absente des données d'entraînement. Les traces doivent montrer quelles ressources ont été utilisées.

Comparaisons appariées : retrieval v3 seul, graphe sans règles, maillage + règles, maillage + règles + opérateurs. Mesures : exact match, taux de réponse non vide, abstention justifiée, précision des dérivations, fausses dérivations, taux de composition G2, couverture, latence, coût, validité des certificats et fuites de données. Les tests unitaires d'un moteur jouet ne sont pas présentés comme résultats du runtime.

Chapitre 7

État de preuve

Une fonctionnalité est « implémentée » uniquement si le code de production, un test qui l'invoque, une trace et une entrée au registre de preuves concordent. « Test vert » signifie uniquement que le contrat exécuté s'est comporté comme spécifié. Les résultats négatifs et abstentions sont conservés. Les capacités non intégrées restent ouvertes ; aucune affirmation de succès ou d'échec général n'est faite à l'avance.

Chapitre 8

Résultats observés et portée des preuves

Les résultats de ce chapitre proviennent des artefacts d'exécution conservés dans le dossier v4. Le jeu de références du laboratoire est lu uniquement par le programme de scoring lancé après la production et le hachage des 66 réponses brutes. Le registre `preuves_eGenBrain_v4.json` relie chaque affirmation aux plages de lignes du code, aux tests et aux reçus.

8.1 Régression logicielle

La suite de régression complète du paquet de développement a terminé avec 136 tests réussis en 17,828 secondes, en configurant `ResourceWarning` comme erreur. Le test maillé ciblé a également passé 21 tests, dont le test adversarial qui modifie le facteur d'unité dans un certificat, recalcule son empreinte puis vérifie que le facteur ne correspond pas au fait primitif stocké. Les tests concernent les contrats du logiciel, pas une capacité cognitive générale.

8.2 Évaluation appariée sur onze micro-mondes

Soit $G = \frac{\#\text{dérivations positives correctes sur cas nécessitant composition}}{\#\text{cas positifs nécessitant composition}}$. Huit des onze cas du laboratoire exigent une composition positive ; trois autres attendent une abstention. Six politiques/modes ont été exécutés sur chaque cas, soit 66 inférences. Les réponses brutes ont été persistées avant l'ouverture du fichier de référence.

Mode	Exact/abstention correcte	Composition positive	G
Retrieval v3 seul	3/11	0/8	0,000
Maillage observations seul	3/11	0/8	0,000
Maillage + primitives	3/11	0/8	0,000
Maillage + primitives + opérateurs	6/11	3/8	0,375
Maillage + primitives + règles	8/11	5/8	0,625
Maillage complet (règles + opérateurs)	11/11	8/8	1,000

Les trois abstentions attendues correspondent à une inversion non autorisée, à l'absence de fait de classe pour le tigre et à une analogie sans correspondance déclarée. Le résultat 11/11 du mode complet est donc composé de 8 dérivations positives et de 3 abstentions correctes ; il ne signifie pas que le moteur peut toujours répondre. Le taux $G = 1$ ne s'applique qu'à ces huit cas positifs explicitement construits. Il ne vaut pas estimation de performance en distribution ouverte.

8.3 Composition numérique hors réponse stockée

Le cas des œufs demande le coût de trois douzaines à 0,50 USD par œuf. Le maillage utilise la primitive observée `unit_scale(dozen,12)`, les observations numériques extraites de la question et

deux applications de l'opérateur général `multiply`. La réponse effective est 18, accompagnée d'un certificat dont la vérification retourne `PASS`. Le cas distinct des poires demande deux douzaines à 1,25 USD par poire ; l'instance utilise une entité et un montant différents et retourne 30 dans le mode complet. Les réponses cibles ne sont pas enregistrées comme faits observés d'entraînement.

Ces réussites ont une portée déterminée : le facteur de douzaine est une primitive explicitement fournie ; la multiplication est un opérateur déterministe déjà inscrit au registre ; le parseur de langue naturelle reconnaît une grammaire bornée « quantité + unité + entité » et « prix par unité ». Les résultats démontrent une composition contrôlée de ces primitives dans ces exemples, et non l'apprentissage autonome de l'arithmétique à partir des seules équations d'entraînement. Le test $2 + 4 = 6$ utilise également l'opérateur générique d'addition ; les exemples $2 + 3 = 5$ et $3 + 4 = 7$ n'autorisent pas à prétendre que l'opérateur a été appris uniquement à partir d'eux.

8.4 Intégration dans le vrai MG hérité

Sur le MG historique reconstruit, la taille est 1 648 779 264 octets et le SHA-256 reste `736a953b0350f1212bf87c2e46dceba1a68b0ff9797d5ec07214bb87526842ea`. La requête des œufs produit 18 en statut `MESH_DERIVED`, avec certificat validé et sans exécuter le repli de retrieval v3. Une requête distincte demandant la capitale de la France a réellement traversé le repli v3 et rendu `NO_ASCR_RESULTS` ; cette réponse vide est conservée comme résultat négatif.

8.5 API, CLI et démarrage sans base

Le test HTTP a passé huit étapes : santé du serveur, racine de l'interface, asset JavaScript, ajout de faits, ajout d'une règle, inférence, interrogation du fait dérivé et chat certifié. Le test CLI a passé six étapes : deux faits, règle, clôture jusqu'au point fixe, requête du fait dérivé et dialogue v4 avec certificat. Un test d'installation à froid avec `-empty-start` a fonctionné sans base préexistante, a créé uniquement un schéma neuf (sans exemples artificiels), et les fichiers SQLite ont réussi `integrity_check` avec zéro violation de clé étrangère.

8.6 Limites actuelles explicitement conservées

— Le parsing du texte naturel est borné ; les requêtes structurées sont la voie la plus explicite et vérifiable. Une affirmation observée exige une provenance d'analyse/entrée.

— Les règles de transitivité, d'inversion, de classe et d'analogie sont déclaratives : elles doivent être ajoutées par l'utilisateur ou le jeu d'apprentissage. Aucune transitivité ou inversion n'est supposée implicitement.

— Les opérateurs arithmétiques sont des primitives déterministes codées de manière générale. La v4 ne démontre pas qu'elle induit ces opérateurs des seuls exemples.

— Le laboratoire A–F comporte onze cas et six modes ; il prouve les résultats des contrats testés dans ces micro-mondes, pas une généralisation à un corpus ouvert ni une théorie complète de l'intelligence.

— Les questions d'induction de règles, de transfert ouvert, de robustesse au paraphrasage et de validité du parseur restent ouvertes et sont listées dans le registre des erreurs du modèle.

Chapitre 9

Commandes de vérification reproductibles

Depuis la racine du projet extrait :

```
python -m egenbrain verify --model model/eGenBrain_v4.sqlite3 --full-check
python -m egenbrain mesh4 stats --mesh-db model/eGenBrain_v4_mesh.sqlite3
python -m egenbrain mesh4 fact --mesh-db model/eGenBrain_v4_mesh.sqlite3 \\
  --subject A --predicate edge --object-json '"B"'
python -m egenbrain mesh4 rule --mesh-db model/eGenBrain_v4_mesh.sqlite3 --input rule.json
python -m egenbrain mesh4 infer --mesh-db model/eGenBrain_v4_mesh.sqlite3
python -m egenbrain mesh4 query --mesh-db model/eGenBrain_v4_mesh.sqlite3 \\
  --pattern-json '{"subject":"A","predicate":"edge","object":"C"}'
python -m egenbrain serve --model model/eGenBrain_v4.sqlite3
```

Les tests et scripts d'évaluation, avec leurs commandes exactes et sorties conservées, sont dans `Amodel_eGenBrain_v4.zip`. Le fichier de suivi porte les checklists, les sources formelles et le registre de preuves.

Chapitre 10

Validation du paquet après installation propre

Les deux ZIP runtime ont été vérifiés avec CRC et chemins sécurisés. Le paquet principal fait 178 000 318 octets et le fragment complémentaire 199 698 071 octets ; chacun reste sous 220 MiB. L'installation a été menée à partir de ces ZIP exacts dans un nouveau dossier vide. Le réassembleur a produit un MG de 1 648 779 264 octets avec SHA-256 `736a953b0350f1212bf87c2e46dceba1a68b0ff9797d5ec07214bb87526842ea` ; `PRAGMA integrity_check` a retourné `ok` et aucune violation de clé étrangère n'a été trouvée.

Le smoke test du runtime installé a interrogé le problème des œufs et obtenu la réponse réelle `18`, statut `MESH_DERIVED`, certificat `PASS` sur deux étapes de composition, et sans repli v3. Un second test, dans le même code installé mais avec un chemin de base inexistant, a exercé `-empty-start` : l'interface et son JavaScript ont répondu, deux faits et une règle ont été ajoutés, une clôture a été exécutée, la conclusion `A edge C` a été interrogée puis renvoyée en chat certifié. Les bases créées à froid passent l'intégrité SQLite et les clés étrangères ; le modèle historique installé n'a pas changé. Ces contrôles démontrent la reproductibilité de l'installation et des chemins exercés, pas une qualité générale supérieure sur tout jeu de données.